

Abschlussprüfung Teil 2

Fachinformatiker Fachrichtung Systemintegration

Realisierung einer automatisierten Bereitstellungspipeline für Windows-Server-Instanzen

durch die Nutzung von Infrastructure as Code und Cloud-Init in einem Unternehmensumfeld

Formale Angaben

Ausbildungsbetrieb:	WIIT AG
Prüfungsteilnehmer:	Moritz Bertz
Projektbetreuer:	Samuel Grossmann und Marco Dragani
Durchführungszeitraum:	21.4.2026 – 26.5.2026
Abgabe bei der IHK:	26.5.2026

Inhaltsverzeichnis

1. Einleitung und betriebliches Projektumfeld	3
1.1 Beschreibung des Ausbildungsbetriebes	3
1.2 Ausgangssituation und Ist-Analyse	3
1.3 Abweichungen vom Projektantrag	4
2. Projektziele und Auftragsbeschreibung/Soll zustand	4
2.1 Projektabgrenzung und Schnittstellen	5
3. Analyse- und Planungsphase	6
3.1 Strategische Make-or-Buy-Entscheidung	6
3.2 Auswahl der Virtualisierungsplattform	6
3.3 Evaluierung der Automatisierungstechnologien	6
3.3.1 Infrastructure-as-Code (IaC) Engine	7
3.3.2 OS-Provisioning & Konfiguration (Bootstrapping)	7
3.4 Wirtschaftlichkeitsbetrachtung und Break-Even-Analyse	9
4. Projektplanung, Ressourcen und Zeitplan	9
4.1 Personal- und Sachmittelplanung	9
4.2 Phasen- und Zeitablaufplanung	10
5. Risikoanalyse und BSI-Grundschutz-Gegenmaßnahmen	11
6. Technische Durchführung und Systemarchitektur	12
6.1 Konfiguration und Audit des Proxmox-Hosts ()	12
6.2 Erstellung und Versiegelung des Windows-Server Golden Images (VM 9002)	14
6.3 Implementierung und Konfiguration von Cloudbase-Init	15
6.4 Systemversiegelung (Sysprep) und Template-Generierung	15
6.5 Infrastruktur als Code (IaC) mit Terraform/OpenTofu & bpg/proxmox	16
6.6 GitLab CI/CD-Pipeline	17
6.7 Automatisierter Active Directory Join per PowerShell Script	18
6.8 Erklärung des automatisierten Bereitstellungsprozesses	19
7. Qualitätssicherung, Testphase und Abnahme	20
7.1 Testprotokoll 1: End-to-End Zero-Touch Deployment	20
7.2 Testprotokoll 2: AD-Join und Client-Erreichbarkeit	21
7.3 Testprotokoll 3: Resilienz- und Rollback-Verfahren	22
8. Fazit und Ausblick	22
8.1 Soll-Ist-Vergleich und Zielerreichung	22
8.2 Abnahmeprotokoll	23
8.3 Lessons Learned und persönliche Reflexion	24
8.4 Ausblick und Skalierungspotenzial	24
9. Anlagenverzeichnis und Anhänge	25
9.1 Technisches Glossar	25
9.2 Literaturverzeichnis	25

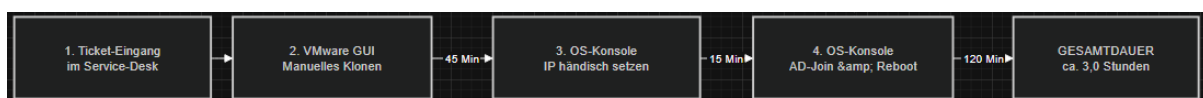
1. Einleitung und betriebliches Projektumfeld

1.1 Beschreibung des Ausbildungsbetriebes

Die WIIT AG präsentiert sich als etablierter Managed Service Provider im Mittelstandsbereich und widmet sich der Bereitstellung von Cloud- und Rechenzentrumslösungen, um ihren B2B-Kunden im geschäftskritischen Betrieb maximale Zuverlässigkeit zu garantieren. Parallel zu anhaltender Expansion und der digitalen Transformation bei den Kundenunternehmen verzeichnet die interne IT-Infrastruktur einen stetig wachsenden Bedarf an virtuellen Server-Kapazitäten. Um diese steigende Anforderung erfüllen zu können, setzt das Unternehmen bereits in verschiedenen Sparten auf die Open-Source-Virtualisierungslösung Proxmox Virtual Environment (VE). Die bereits vorhandenen Kompetenzen sollen in diesem Projekt genutzt werden, um eine zeitgemäße, automatisierte Deployment-Pipeline zu schaffen und bestehende Hypervisor-Kapazitäten ohne weitere Lizenzausgaben in Betrieb zu nehmen.

1.2 Ausgangssituation und Ist-Analyse

Die heutige Bereitstellung von Windows-Server-Instanzen basiert auf einem über Jahre gewachsenen Verwaltungsprozess mit stark manuellen Elementen. Sobald ein Fachbereich eine neue Instanz benötigt, wird ein entsprechendes Ticket im Service-Desk registriert. Danach muss ein Systemadministrator händisch eine überholte Vorlage aus der existierenden Proxmox-Infrastruktur duplizieren und im Anschluss aufwändige Folgearbeiten erledigen: Umbenennung des Computers, Festlegung der Netzwerkparameter, Aufbau des Active-Directorys nach Kundenwunsch. Mit diesem Ablauf ist pro Instanz ein Zeitaufwand von etwa drei Arbeitsstunden verbunden, wodurch wertvolle Verwaltungskapazitäten gebunden werden. Hinzu kommt das Problem des Configuration Drift: Die verschiedenen Umgebungen – Test, Staging und Produktion – weisen aufgrund individueller manueller Änderungen subtile Unterschiede auf, was später bei der Auslieferung von Software zu schwer nachvollziehbaren Problemen führen kann.



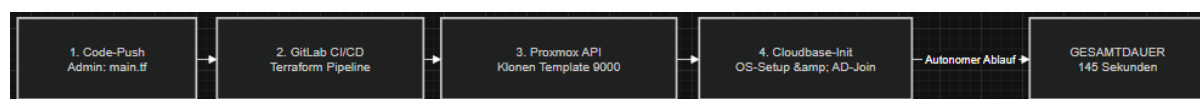
1.3 Abweichungen vom Projektantrag

Nutzung einer bestehenden Proxmox-Testumgebung

Abweichend vom ursprünglichen im Projektantrag vorgesehenen persönlichen Aufbaus, hat man darauf verzichtet, einen unabhängigen physischen Server aufzubauen oder Proxmox neu zu installieren. Nach Projektgenehmigung entschied man sich pragmatisch dafür, die Pipeline stattdessen auf einem vorhandenen Proxmox-Knoten der WIIT AG zu realisieren, der ohnehin ausschließlich zu Testzwecken genutzt wird. Begründung: Diese Entscheidung fiel vor allem aus ökonomischen Gründen und um Risiken zu minimieren. Eine bereits bestehende und von der Produktionsumgebung isolierte Testumgebung bot die perfekte Basis, um die Bereitstellungspipeline risikofrei zu entwickeln, ohne den laufenden Betrieb zu beeinträchtigen. Darüber hinaus hätte die Anschaffung, Verkabelung und Grundkonfiguration eines zusätzlichen Servers wertvolle Zeit aus dem knappen 40-Stunden-Budget gekostet. Auf diese Weise ließ sich die Projektarbeit vollständig auf das eigentliche technische Ziel konzentrieren nämlich auf die deklarative Automatisierung mittels Infrastructure as Code und Cloudbase-Init ohne inhaltliche Kompromisse eingehen zu müssen.

2. Projektziele und Auftragsbeschreibung/Soll zustand

Im Mittelpunkt dieser Projektarbeit steht die Entwicklung, Analyse und Umsetzung einer vollkommen automatisierten Bereitstellungspipeline für Windows-Server-Instanzen. Hierbei erfolgt ein Wechsel weg von der klassischen, manuellen Verwaltungspraxis hin zu einem deklarativen Infrastructure-as-Code-Ansatz (IaC). Die Infrastruktur wird dabei in Code abgebildet, was sicherstellt, dass die Bereitstellung konsistent, wiederholbar und zuverlässig abläuft.



Die einzelnen Ziele umfassen folgende Punkte:

- **Technologieumstellung:** Aufbau der Pipeline mit Proxmox VE einer kostenlosen Alternative zu VMware.
- **Netzwerktrennung:** Etablierung einer netzwerkseitigen Isolation durch eine dedizierte virtuelle Bridge (vbr1, VLAN 2663).
- **Zero-Touch-Bereitstellung:** Anfertigung eines verschlossenen Windows Server Golden Images mittels Microsoft Sysprep.
- **GitOps-Management:** Steuerung aller Parameter durch versionskontrollierte Skripte (GitLab). Integration von Cloudbase-Init zur Konfiguration von Windows-Systemen.
- **Automatisierter Domänenbeitritt:** Ausführung eines PowerShell-Skripts während des Boot-Vorgangs für die eigenständige Eingliederung ins Active Directory.
- **Wirtschaftlichkeitsnachweis:** Darlegung der Amortisationsdauer sowie Gewährleistung der Einsatzbereitschaft durch normierte Testverfahren.

2.1 Projektabgrenzung und Schnittstellen

• Eric Betzer

Infrastruktur Team Schnittstelle: Verantwortlich für die Verbindung zwischen Proxmox und GitLab sowie die Einrichtung von GitLab-Zugangsdaten und Token-Generierung für das vorliegende Projekt.

• Marco Dragani

Proxmox Team Schnittstelle: Mein Ansprechpartner bei der praktischen Umsetzung des Projekts und für die Erstellung des entsprechenden Proxmox-Benutzers in der Testumgebung zuständig.

• Samuel Großmann

Abteilungsleitung führt grundlegende Gespräche zum Projektablauf, bestimmt die einzelnen Ansprechpartner für mein Vorhaben.

Im Mittelpunkt dieses Projekts steht ausschließlich die Bereitstellung grundlegender Infrastrukturkomponenten auf Betriebssystemebene (OS-Provisioning). Alles Weitere wie die Installation konkreter Anwendungen fällt nicht in den Projektumfang und wird als nachgelagerter Schritt (Configuration Management) behandelt. Das Projekt endet formal dann, wenn mittels IaC-Codes eine funktionsfähige Virtuelle Maschine erstellt ist, die im Netzwerk erreichbar und in die Unternehmensdomäne integriert wurde und sich via RDP beziehungsweise PowerShell Remoting (WinRM) administrieren lässt.

Darüber hinaus bestehen Schnittstellen zu den zuständigen Netzwerkadministratoren des Infrastruktur Teams (VLAN-Aktivierung auf den physischen Cisco Core-Switches) sowie zu First- und Second-Level-Support. Ein Administrationshandbuch wird als Dokumentation für den Kunden verfasst. Die Gesamtdauer des Projekts beträgt gemäß IHK-Vorgaben 40 Stunden.

3. Analyse- und Planungsphase

3.1 Strategische Make-or-Buy-Entscheidung

Bevor man sich intensiver mit einzelnen Hypervisor-Systemen beschäftigte, stand zunächst eine strategische Make-or-Buy-Analyse auf dem Programm. Die Option, eine fertige kommerzielle Lösung für das Lifecycle-Management zu erwerben, kam nicht in Frage, da die laufenden Lizenzgebühren hätten sich ebenso negativ ausgewirkt wie der erhebliche Aufwand zur Anbindung an die bestehende Netzwerkinfrastruktur. Beide Faktoren zusammen führten zu einer wirtschaftlich ungünstigen Gesamtsituation. Demgegenüber bietet der Aufbau einer eigenen Pipeline (Make) mit Open-Source-Technologien der WIIT AG entscheidende Vorteile: Das Unternehmen bleibt Herr über seinen Code, umgeht die Abhängigkeit von einem einzelnen Anbieter und kann dabei auf die vorhandene Erfahrung im Bereich Linux-Systemintegration zurückgreifen.

3.2 Auswahl der Virtualisierungsplattform

Für einen aussagekräftigen Vergleich der drei Virtualisierungslösungen Proxmox VE, VMware vSphere und Microsoft Hyper-V wurde eine umfassende Nutzwertanalyse erstellt. Die Bewertungskriterien basieren auf den strategischen Anforderungen der IT-Leitung und wurden entsprechend gewichtet. Zur quantitativen Bewertung kam eine Skala von 1 (ungenügend) bis 10 (optimal) zum Einsatz.

Bewertungskriterium	Gewichtung	VMware vSphere	Microsoft Hyper-V	Proxmox VE
Lizenz- und Betriebskosten	35 %	2 (0,70)	6 (2,10)	10 (3,50)
IaC-Automatisierung & API	25 %	9 (2,25)	5 (1,25)	9 (2,25)
Storage & Network Performance	20 %	9 (1,80)	8 (1,60)	9 (1,80)
Integrierte Backup-Lösungen	10 %	6 (0,60)	5 (0,50)	10 (1,00)
Einarbeitungsaufwand / Expertise	10 %	9 (0,90)	8 (0,80)	5 (0,50)
Gewichteter Gesamtwert	100 %	6,05 Punkte	6,25 Punkte	9,05 Punkte

Tabelle 1: Nutzwertanalyse der Virtualisierungsplattformen (Skala 1–10, gewichtete Punkte in Klammern).

Aus der quantitativen Bewertung ergibt sich unmissverständlich, dass Proxmox VE die bessere Wahl darstellt. Besonders bei den Lizenz- und Betriebsausgaben zeigt sich die Open-Source-Lösung konkurrenzlos, da die optionalen Enterprise-Support-Verträge kosten nur einen Bruchteil dessen, was für vSphere Foundation Lizenzen zu zahlen wäre. Hinsichtlich

Infrastructure-as-Code-Automatisierung bieten sowohl VMware als auch Proxmox hochwertige RESTful APIs und offizielle Terraform-Provider an; hinzu kommt, dass Proxmox darüber hinaus Cloud-Init-Laufwerke nativ unterstützt. Bei der Verarbeitung netzwerkintensiver Last erreicht Proxmox durch seine KVM-basierte Architektur erheblich bessere Durchsatzraten. Der minimal erhöhte Aufwand für die Einarbeitung wird durch das im Unternehmen bereits vorhandene Proxmox-Know-how problemlos kompensiert.

3.3 Evaluierung der Automatisierungstechnologien

Um sicherzustellen, dass die Wahl der passenden Technologien auf solider Grundlage, transparent und objektiv erfolgt, haben wir zunächst umfassende Nutzenanalysen durchgeführt. Unsere Bewertungskriterien richten sich nach den individuellen Anforderungen des Projekts und wurden entsprechend priorisiert hierzu gehören Zero-Touch-Deployment, die Kompatibilität mit GitOps, der notwendige Infrastruktur-Overhead sowie wirtschaftliche Effizienz und Ressourcenschonung.

3.3.1 Infrastructure-as-Code (IaC) Engine

Um virtuelle Maschinen deklarativ bereitzustellen, war zunächst eine Bewertung der verfügbaren IaC-Engines notwendig. Angesichts der Lizenzierungsänderungen bei HashiCorp (Wechsel zur BSL) haben wir Terraform und den Open-Source-Fork OpenTofu miteinander verglichen:

Bewertungskriterium	Gewichtung	Terraform	OpenTofu
Lizenzmodell & Vendor Lock-in	40 %	3 (1,2)	10 (4,00)
Kompatibilität & Migration	30 %	10 (3,00)	9 (2,7)
Feature-Set & Performance	30 %	10 (3,00)	10 (3,00)
Gewichteter Gesamtwert	100 %	7,20 Punkte	9,70 Punkte

Tabelle 2: Nutzwertanalyse der Infrastructure as Code Engine (Skala 1–10, gewichtete Punkte in Klammern).

Begründung der Entscheidung:

Die Zahlen sind eindeutig: Mit 9,70 Punkten liegt OpenTofu deutlich vor Terraform, das nur 7,20 Punkte erreicht. Das zentrale Anliegen des Projekts ist es, sich von Herstellerabhängigkeiten zu befreien, daher wirkt sich OpenTofu's freies Lizenzmodell (MPL 2.0) mit 40 % Gewichtung stark positiv aus. Hinzukommt, dass OpenTofu vollständig mit

bestehenden Terraform-Providern und -Modulen kompatibel ist das ermöglicht problemlose Migrationen ohne Funktionsverluste.

3.3.2 OS-Provisioning & Konfiguration (Bootstrapping)

Nachdem die IaC-Engine die virtuelle Hardware zur Verfügung gestellt hat, muss das Betriebssystem (Windows Server) vollkommen autark und ohne jegliche manuelle Eingriffe starten können. Um die geeignetste Methode zu ermitteln, wurden drei technische Ansätze einer vergleichenden Analyse unterzogen:

Bewertungskriterium	Gewichtung	Cloudbase-Init	MS WDS / MDT	Ansible
Zero-Touch & Bootstrapping	40 %	10 (4,00)	5 (2,00)	2 (0,80)
Infrastruktur-Overhead	30 %	9 (2,70)	3 (0,90)	8 (2,40)
GitOps-Kompatibilität	30 %	10 (3,00)	2 (0,60)	9 (2,70)
Gewichteter Gesamtwert	100 %	9,70 Punkte	3,50 Punkte	5,90 Punkte

Tabelle 3: Nutzwertanalyse der OS-Provisioning & Konfiguration (Bootstrapping) (Skala 1–10, gewichtete Punkte in Klammern).

Begründung der Entscheidung:

Mit einem hervorragenden Ergebnis von 9,70 Punkten erweist sich Cloudbase-Init als die ideale Lösung zum OS-Bootstrapping.

- **Microsoft WDS/MDT (3,50 Punkte)** scheidet aus, weil es massive infrastrukturelle Anforderungen mit sich bringt (PXE-gestützte Netzwerkkumgebung, DHCP-Konfigurationen, separater Windows-Bereitstellungsserver) und gleichzeitig keine brauchbare deklarative GitOps-Unterstützung bietet.
- **Ansible (5,90 Punkte)** kann seine Stärken als Konfigurations-Werkzeug erst nutzen, wenn bereits ein lauffähiges Betriebssystem mit aktiver Netzwerkverbindung und eingerichtetem WinRM/SSH-Service existiert (klassisches Henne-Ei-Problem).
- **Cloudbase-Init (9,70 Punkte)** durchbricht dieses Dilemma auf elegante Weise: Als lokaler Windows-Dienst liest es beim ersten Hochfahren die Metadaten aus, die Proxmox über ein virtuelles CD-ROM-Laufwerk (ConfigDrive2) bereitstellt, lokal und offline ein. Dadurch wird ein echtes, sicheres Zero-Touch-Deployment möglich.

3.4 Wirtschaftlichkeitsbetrachtung und Break-Even-Analyse

Da das Projekt ausschließlich auf vorhandener Hardware sowie kostenloser Open-Source-Software basiert, reduzieren sich die erforderlichen Investitionen auf die interne Arbeitszeit der Mitarbeiter. Bei der WIIT AG beträgt das Stundenhonorar eines Systemadministrators 65,00 €.

- Gesamte Anfangsinvestition: $40 \text{ h} \times 65,00 \text{ €/h} = 2.600,00 \text{ €}$
- Manuelle Kosten pro Bereitstellung (Kman): $3,0 \text{ h} \times 65,00 \text{ €/h} = 195,00 \text{ €}$
- Automatisierte Kosten pro Bereitstellung: $0,25 \text{ h} \times 65,00 \text{ €/h} = 16,25 \text{ €}$
- Einsparungen je Deployment: $195,00 \text{ €} - 16,25 \text{ €} = 178,75 \text{ €}$
- Amortisationsmenge: $2.600,00 \text{ €} / 178,75 \text{ €} \approx 14,54 \text{ Einheiten}$

Bereits beim 15. Server wird sich die Investition amortisiert haben. Angesichts eines monatlichen Durchschnitts von etwa zehn neu hinzukommenden Servern ist die Gewinnschwelle schon nach rund anderthalb Monaten überschritten. Nicht zu unterschätzen sind auch die nahezu vollständig gesunkene Fehlerquote und die Tatsache, dass Entwickler ihre Testsysteme nun in wenigen Minuten anstelle von Stunden bereitstellen können.

4 Projektplanung, Ressourcen und Zeitplan

4.1 Personal- und Sachmittelplanung

Der Projektleiter fungierte als Hauptverantwortlicher für die praktische Umsetzung; der Projektbetreuer konzentrierte sich dagegen primär auf die fachliche Evaluation der erzielten Resultate. Für die technische Ausstattung der WIIT AG wurden nachfolgende Komponenten herangezogen:

- **Hardware:** Ein Test Proxmox Server (32 x AMD EPYC 7302 16-Core Processor, 256 GB RAM, 3,7 TB zpool ZFS für Daten) diene als Proxmox-Host; ein DELL Latitude 3450 stand für administrative Tätigkeiten bereit.
- **Software:** Im Softwarebereich kamen Proxmox VE 8.x, Installationsmedien für Windows Server 2022/2025, das Fedora VirtIO-Treiberpaket (v0.1.285), Cloudbase-Init (Continuous Build MSI v1.1.9.dev2), GitLab Server, Visual Studio Code sowie eine Debian VM auf dem Proxmox als GitLab Runner Server zum Einsatz.
- **Netzwerk:** Auf den physischen Core-Switches wurde ein dediziertes Bereitstellungs-VLAN (2663) verwendet.

4.2 Phasen- und Zeitablaufplanung

Das Projekt folgte dem klassischen Wasserfallmodell, weil die einzelnen technischen Schritte aufeinander aufbauen und zwingend in einer bestimmten Reihenfolge absolviert werden mussten.

Projektphase / Arbeitspaket	Geplant (h)	Ist (h)	Abw. (h)
Phase 1: Analyse und Planung	8,0	8,0	0,0
Kick-off-Meeting, Zieldefinition und Anforderungsanalyse	1,5	1,5	0,0
Ist-Analyse VMware-Prozess & Risikoanalyse	2,0	2,0	0,0
Nutzwertanalysen (Hypervisor & Automatisierungstool)	2,5	2,5	0,0
Wirtschaftlichkeitsbetrachtung und ROI-Kalkulation	2,0	2,0	0,0
Phase 2: Realisierung und Implementierung	18,0	19,5	+1,5
Netzwerkkonfiguration: Proxmox Einstellungs-Audit & VLAN/Bridge-Isolation	3,0	3,0	0,0
VM-Setup (VM 9002), Windows Server Setup & VirtIO-Treiber	3,5	3,0	-0,5
Installation, Anpassung & Troubleshooting Cloudbase-Init & QEMU Agent	2,5	4,5	+2,0
Systemversiegelung (Sysprep / Unattend.xml) & Template-Erstellung	3,0	3,0	0,0
Git-Repository-Strukturierung & Erstellung der YAML-Dateien	3,0	3,0	0,0
PowerShell-Entwicklung für automatisierten AD-Beitritt	3,0	3,0	0,0
Phase 3: Testphase und Qualitätssicherung	6,0	4,5	-1,5
Zero-Touch Deployment-Tests (Klonvorgänge, Hostname, IP)	2,5	2,0	-0,5
Validierung AD-Join und administrative Rechteprüfung	1,5	1,5	0,0
Resilienz-Tests: Git-Rollback bei simulierter Fehlkonfiguration	2,0	1,0	-1,0
Phase 4: Projektabschluss und Dokumentation	8,0	8,0	0,0
Erstellung der Kundendokumentation (Administrationshandbuch)	2,5	2,5	0,0
Erstellung und Einbindung des Netzwerk-Topologieplans	1,0	1,0	0,0

Projektphase / Arbeitspaket	Geplant (h)	Ist (h)	Abw. (h)
Verfassen der formalen IHK-Projektdokumentation	3,5	3,5	0,0
Projektanbahnung und Übergabe an den Auftraggeber	1,0	1,0	0,0
Gesamtaufwand	40,0	40,0	0,0

Tabelle 4: Geplanter versus tatsächlicher Projektablauf in Stunden.

Bei der Integration von Cloudbase-Init zeigten sich erhebliche Kompatibilitätsprobleme in der Metadaten-Verarbeitung (Bug ID 4493), weshalb eine detaillierte Analyse der Logdateien notwendig wurde und letztendlich 2,0 Stunden zusätzlicher Aufwand entstand. Allerdings machte sich in der anschließenden Testphase die verbesserte Zuverlässigkeit des Systems deutlich positiv bemerkbar und führte zu Einsparungen von 1,5 Stunden. Dadurch ergibt sich insgesamt eine relative Gesamtabweichung von genau 0 %.

5. Risikoanalyse und BSI-Grundschutz-Gegenmaßnahmen

Im ersten Schritt wurde eine quantitative Risikoanalyse nach den Richtlinien des BSI-IT-Grundschutzes vorgenommen. Ziel war es, potenzielle Gefährdungen strukturiert zu identifizieren und anschließend angemessene Schutzmaßnahmen festzulegen.

Nr.	Projektrisiko	EW (%)	Mögliche Auswirkungen	Geplante Gegenmaßnahmen
1	Broadcast-Leaking im Produktivnetz (rogue DHCP)	40 %	Kritisch: Konflikte bei IP-Adressen oder Störungen in den produktiven Systemen der WIIT AG.	Konfiguration einer isolierten Linux Bridge (vbr1). VLAN-Tagging auf Core-Switches (VLAN 2663).
2	Unvereinbarkeit von Cloudbase-Init mit dem Proxmox-Metadatenservice (Bug ID 4493)	60 %	Hoch: Zero-Touch funktioniert nicht, es ist manuelle Eingabe nötig.	Verwendung des aktuellen Continuous Build (v1.1. 9.dev2). Manuelle Aktivierung des ProxmoxMetadataService.
3	Fehler bei Sysprep wegen blockierter Windows AppX-Pakete	30 %	Mittel: Zeitverlust durch die Wiederherstellung des Images.	Bereinigung von Bloatware mit PowerShell vor Sysprep. Snapshot erstellen vor der Ausführung von Sysprep.

Tabelle 5: Risikoanalyse vor Projektbeginn (EW = Eintrittswahrscheinlichkeit).

6 Technische Durchführung und Systemarchitektur

6.1 Konfiguration und Audit des Proxmox-Hosts ()

Zu Beginn der technischen Realisierung wurde auf dem Proxmox-Knoten in detailliertes Pre-Flight-Audit durchgeführt. Dabei prüfte man die SSH-Verbindung, validierte die notwendigen Root-Privilegien für die qm-CLI-Operationen und bestätigte, dass die Hardware-Virtualisierung funktionsfähig ist.

```
egrep -c '(vmx|svm)' /proc/cpuinfo
# Ausgabe: 128 (Virtualisierung aktiv)
```

Der erste notwendige Schritt besteht darin, im lokalen Proxmox-Speicherpool den Inhaltstyp 'Snippets' zu aktivieren. Dies ist absolut erforderlich. Standardmäßig unterbindet Proxmox das Speichern von Skripten. Solange man diese Freigabe nicht manuell über die grafische Oberfläche vornimmt (Datacenter → Storage → Edit → Content: Snippets), funktionieren die automatisierten Skript-Injektionen nicht. Um die IT-Sicherheitsrichtlinien für Layer-2-Isolation einzuhalten, wurde die Netzwerkkonfiguration des Hypervisors angepasst (/etc/network/interfaces):

```
auto vmbr0
iface vmbr0 inet static
    address -----
    gateway -----
    bridge-ports enolnp0
    bridge-stp off
    bridge-fd 0

auto vmbr1
iface vmbr1 inet manual
    bridge-ports eno2np1
    bridge-stp off
    bridge-fd 0
    bridge-vlan-aware yes
    bridge-vids 2-4094
```

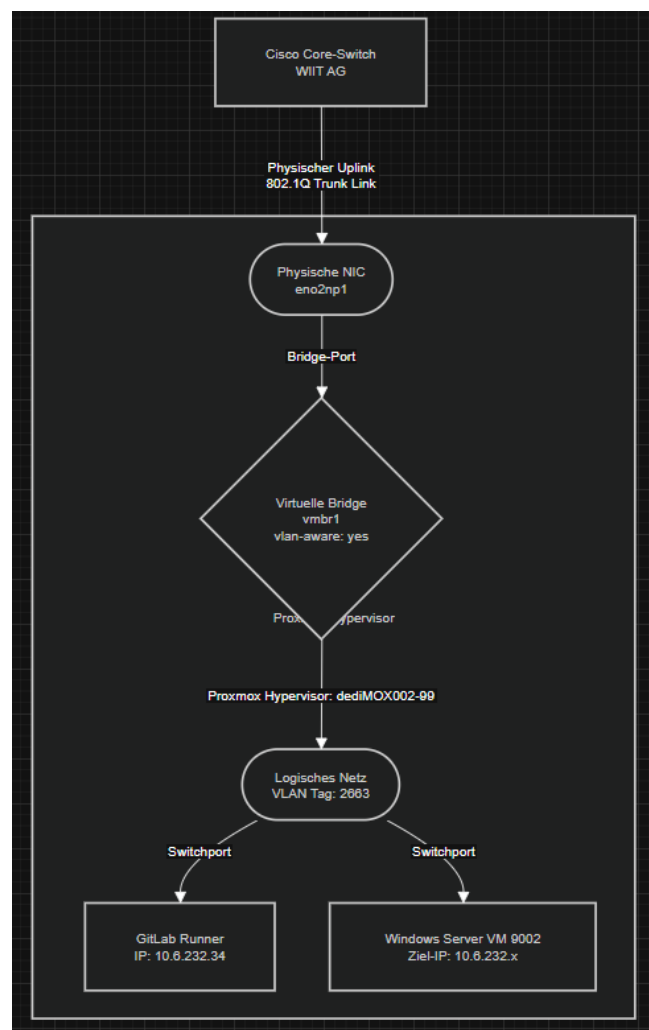
Listing 1: Netzwerkkonfigurationsdatei /etc/network/interfaces des Proxmox-Knotens.

Netzwerkschnittstellen:

Schnittstelle / Netz	Typ	Konfiguration	Funktion / Beschreibung
eno2np1	Physischer Uplink	802.1Q Trunk	Physische Netzerkennung des Hypervisors () an die Switch-Infrastruktur der WIIT AG.
Vmbr1	Virtuelle Bridge	vlan-aware: yes	Dedizierter Layer-2-Switch zur strikten logischen Trennung vom Management-Netzwerk.
VLAN 2663	Logisches Netz	Tag: 2663	Kapselung der Broadcast-Domain. Beherbergt die Test-VMs sowie die CI/CD-Infrastruktur (GitLab Runner: 10.6.232.34).
Eth0(VM)	Virtueller Adapter	10.6.232.x	Paravirtualisierte VirtIO-Netzwerkschnittstelle der provisionierten Windows-Instanz.

Tabelle 6: Netzwerkschnittstellen

In der Proxmox-GUI ließ sich die virtuelle Bridge vmbr1 2663 als logische Partitionierungsebene einbinden, wodurch die Test-VMs eine isolierte Switch-Infrastruktur ohne physischen Uplink nutzen konnten. Dies verhinderte zuverlässig, dass Broadcast-Lecks wie etwa rogue DHCP-Requests in das Produktionskernnetz der WIIT AG gelangten.



Name	Alternative Names	Type	Active	Autostart	VLAN a...	Ports/Slaves	Bond Mode	CIDR	Gateway	Comment
eno1np0	enp202s0f0np0 enx3cecaf724686	Network Device	Yes	No	No					
eno2np1	enp202s0f1np1 enx3cecaf724687	Network Device	Yes	No	No					
enp129s0f...	enx7cfe9040a47a	Network Device	No	No	No					
enp129s0f...	enx7cfe9040a47b	Network Device	No	No	No					
vmbr0		Linux Bridge	Yes	Yes	No	eno1np0				
vmbr1		Linux Bridge	Yes	Yes	Yes	eno2np1				VLAN
vmbr1.2663		Linux VLAN	Yes	Yes	No			10.6.232.33/24		Projekt-Moritz_Bertz

6.2 Erstellung und Versiegelung des Windows-Server Golden Images (VM 9002)

Die Basis-VM mit der ID 9002 war die Grundlage für die spätere Erstellung des Golden Images. Um optimale I/O-Leistung zu sichern, wurden paravirtualisierte Treiber verwendet:

- Maschinentyp: Q35 mit OVMF (UEFI) Firmware.
- SCSI-Controller: VirtIO SCSI single. Durch diesen Controller entfallen die typischen Emulationskosten von IDE/SATA, und TRIM-Befehle (Discard) werden unterstützt. • Festplatte: 80 GiB Raw Disk im lokalen ZFS-Pool, dazu kam aktive SSD-Emulation, Discard-Option sowie ein dedizierter I/O-Thread.
- CPU & Memory: 4 Kerne (Type: host) mit aktivierten CPU-Mitigationsflags und 8.192 MiB RAM. Weil das Standard-Installationsmedium von Windows ohne paravirtualisierte Treiber ausgeliefert wird, war die virtuelle Festplatte anfangs nicht sichtbar. Die Fedora VirtIO-Treiber-ISO (virtio-win-0.1.285.iso) wurde als zusätzliches CD-Laufwerk geladen, um Abhilfe zu schaffen. Im Setup-Menü von Windows ließ sich der Treiber dann über D:\amd64\2k22\vioscsi.inf laden.

Nach erfolgreicher Installation des Betriebssystems folgte gleich darauf die Einrichtung des QEMU Guest Agents. Dadurch entsteht eine zuverlässige Kommunikation für Systembefehle (etwa ACPI-Shutdowns) zwischen Hypervisor und Gast-OS.

6.3 Implementierung und Konfiguration von Cloudbase-Init

Cloudbase-Init stellt das Herzstück unseres Zero-Touch Deployment-Ansatzes dar. Bei ersten Versuchen mit der stabilen Version 1.1.4 traten erhebliche Kompatibilitätsprobleme mit dem Proxmox-ConfigDrive-Format auf (Bug ID 4493): Das Passwort und der Hostname wurden nicht berücksichtigt, da das JSON-Parsing nicht ordnungsgemäß funktionierte. Um dieses Hindernis zu überwinden, griffen wir auf die neueste Continuous Build-Variante (Cloudbase-Init 1.1.9.dev2) zurück. Danach wurden die Konfigurationsdateien unter C:\Program Files\Cloudbase Solutions\Cloudbase-Init\conf\ umfassend neu strukturiert:

```
[DEFAULT]
username=Administrator
groups=Administratoren
first_logon_behaviour=no
inject_user_password=true
config_drive_raw_hhd=true
config_drive_cdrom=true
config_drive_vfat=true
metadata_services=cloudbaseinit.metadata.services.proxmox.ProxmoxMetadataService,
    cloudbaseinit.metadata.services.configdrive.ConfigDriveService
plugins=cloudbaseinit.plugins.common.sethostname.SetHostNamePlugin,
    cloudbaseinit.plugins.windows.setuserpassword.SetUserPasswordPlugin,
    cloudbaseinit.plugins.common.userdata.UserDataPlugin
allow_reboot=false
stop_service_on_exit=false
verbose=true
debug=true
log_dir=C:\Program Files\Cloudbase Solutions\Cloudbase-Init\log\
log_file=cloudbase-init.log
```

Listing 2: Konfigurationsdatei cloudbase-init.conf.

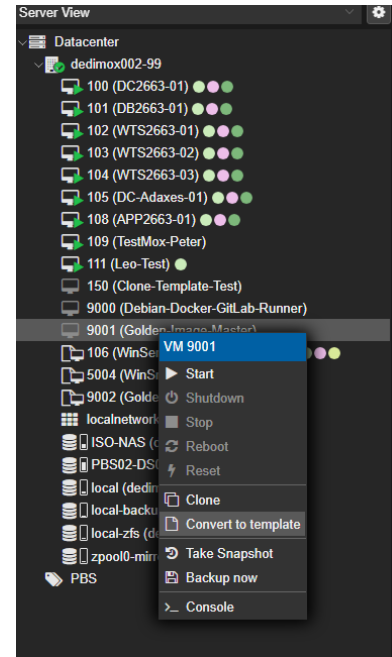
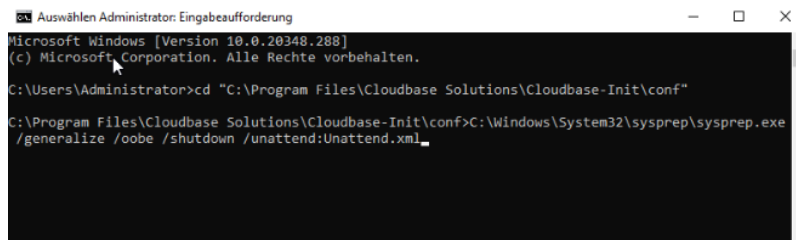
Damit Cloudbase-Init das ConfigDrive-Dateisystem zuverlässig durchsucht, wird der ProxmoxMetadataService explizit konfiguriert. Die beiden Plugins SetHostNamePlugin und SetUserPasswordPlugin sorgen dafür, dass der Computernamen und das Administratorpasswort direkt beim ersten Systemstart neu gesetzt werden. Für die Ausführung von Post-Provisioning-Skripten etwa den Active-Directory-Join braucht man das UserDataPlugin.

6.4 Systemversiegelung (Sysprep) und Template-Generierung

Um sicherzustellen, dass Klone problemlos ins Active Directory integriert werden können ohne Netzwerkstörungen oder doppelte SIDs - haben wir die VM mit Microsoft Sysprep generalisiert. Über eine speziell konfigurierte Unattend.xml-Antwortdatei wird Windows angewiesen, alle OOBE-Dialoge zu überspringen und direkt den Cloudbase-Init-Service zu starten. Der Sysprep-Befehl wurde mit administrativen Berechtigungen ausgeführt:

```
C:\Windows\System32\sysprep\sysprep.exe /generalize /oobe /shutdown
/unattend:Unattend.xml
```

Nachdem die VM eigenständig herunterfuhr, konnte ihr Status über die Proxmox-Oberfläche per Rechtsklick - Convert to Template (VM-ID 9002) in ein schreibgeschütztes Template umgewandelt werden. Damit war das Golden Image fertiggestellt.



6.5 Infrastruktur als Code (IaC) mit Terraform/OpenTofu & bpg/proxmox

Zur vollständig deklarativen Verwaltung der Infrastruktur kommt der Provider bpg/proxmox zum Einsatz. In der praktischen Anwendung zeigte sich, dass das einfache Klonen eines Templates (VM-ID 9002) zu sogenannten Konfigurations-Drifts führen kann. Diese treten etwa auf, wenn das Template ein veraltetes VLAN-Tag oder ein defektes Hardware-Laufwerk mitbringt. Um derartige Probleme zu vermeiden, wurde der IaC-Code entsprechend aufgebaut, um netzwerkspezifische Parameter wie die VLAN-ID gezielt zu überschreiben. Ein weiteres Problem ergab sich bei der dynamischen Erzeugung des Cloud-Init-Mediums: Der Proxmox-Provider erfordert für die Erstellung des ConfigDrive2-ISOs notwendigerweise einen physisch existierenden und freigegebenen Datastore. Da der ursprüngliche lokale LVM-Speicher auf dem Node nicht zur Verfügung stand, wurde die `datastore_id` bewusst auf den redundanten ZFS-Mirror (`zpool0-mirror0`) gelegt.

```
resource "proxmox_virtual_environment_vm" "windows_server" {
  name = "DC-1010"

  node_name = ""

  vm_id = 123

  started = true

  clone {
```



```
vm_id = 9002
}

# Explizites Überschreiben der Template-Netzwerkkarte zur Isolation
network_device {
  bridge = "vmbr1"
  vlan_id = 2663
}

initialization {
  # Zuweisung auf den ausfallsicheren ZFS-Mirror
  datastore_id = "zpool0-mirror0"
  interface = "ide2"
  ip_config {
    ipv4 {
      address = "10.6.232.200/24"
      gateway = "10.6.232.1"
    }
  }
  user_account {
    username = "Administrator"
    password = var.vm_admin_password
  }
  user_data_file_id = "local:snippets/ad-join.yaml"
}
}
```

Listing 3: Terraform-Konfigurationsdatei main.tf (bpg/proxmox-Provider, gekürzt).

6.6 GitLab CI/CD-Pipeline

Eine knifflige technische Aufgabe ergab sich bei der Authentifizierung des OpenTofu-Runners gegenüber dem GitLab HTTP-Backend. Der vom System automatisch generierte `$CI_JOB_TOKEN` besitzt nur für die Laufzeit eines bestimmten Jobs (etwa plan) Gültigkeit, sodass die anfängliche Weitergabe des Tokens beim Befehl `tofu init` später in der apply-Phase zu Authentifizierungsfehlern führte ("HTTP remote state endpoint requires auth"). Das

Problem ließ sich dadurch beheben, dass man die Backend-Anmeldedaten komplett von den CLI-Befehlen abtrennte und sie stattdessen mittels der globalen Umgebungsvariablen `TF_HTTP_USERNAME` und `TF_HTTP_PASSWORD` in den Arbeitsspeicher des Shell-Runners schleuste wobei `TF_HTTP_PASSWORD` ein vom Betriebssystem regelmäßig rotiertes Secret darstellt. Auf diese Weise kann OpenTofu die erforderlichen Zugangsdaten in jeder Pipeline-Etappe neu abholen und verwendet stets einen aktuellen, gültigen Token, ohne verbrauchte Login-Informationen in der State-Datei zu speichern.

CI/CD Variables </> 4 Reveal values Add variable			
Key ↑	Value	Environments	Actions
AD_JOIN_PASSWORD  Protected Masked Expanded	 	All (default) 	 
PROXMOX_TOKEN_SECRET  Protected Masked Expanded	 	All (default) 	 
PROXMOX_VE_ENDPOINT  Protected Masked Expanded	 	All (default) 	 
VM_ADMIN_PASSWORD  Protected Masked Expanded	 	All (default) 	 

6.7 Automatisierter Active Directory Join per PowerShell Script

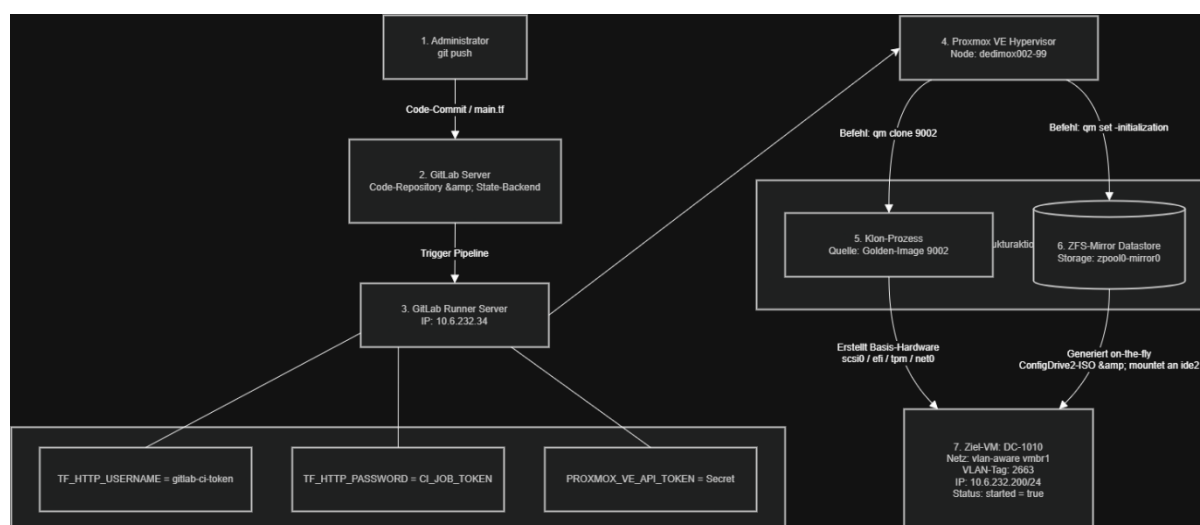
Für die automatisierte Domänenintegration war es notwendig, ein PowerShell-Skript zu erstellen und dieses als YAML-Snippet im Proxmox-Snippet-Speicher (`/var/lib/vz/snippets/ad-join.yaml`) zu hinterlegen:

```
#cloud-config
write_files:
  - content: |
      $secpasswd = ConvertTo-SecureString ${domain_password} -AsPlainText -Force
      $mycreds = New-Object System.Management.Automation.PSCredential ("wiit-iac.local\JoinAccount", $secpasswd)
      Add-Computer -DomainName "wiit-iac.local" -Credential $mycreds \
        -OUPath "OU=Servers,OU=ManagedDevices,DC=wiit-iac,DC=local" -Force -Restart
      path: 'C:\Program Files\Cloudbase Solutions\Cloudbase-Init\LocalScripts\ad-join.ps1'
```

Listing 4: YAML-Snippet `ad-join.yaml` zur Injektion des Domänenbeitritts-Skripts.

Cloudbase-Init greift auf das vom UserDataPlugin bereitgestellte Snippet zu, schreibt die verschlüsselten Anweisungen in eine lokale PowerShell-Datei auf dem neuen Server und führt sie anschließend mit administrativen Rechten aus. Mit einem äußerst restriktiv konfigurierten Service-Account (JoinAccount) lässt sich das Sicherheitsrisiko erheblich minimieren.

6.8 Erklärung des automatisierten Bereitstellungsprozesses



Screenshot: Architekturdiagramm

Das Diagramm zeigt, wie die automatisierte Bereitstellung und das Domänenbeitrittsskript zeitlich und logisch ablaufen. Den Anfang macht ein deklarativer Code-Push auf Administratorebene. Nach einer Validierung im Versionskontrollsystem übernimmt der GitLab-Runner (IP: 10.6.232.34) die Orchestrierung. Um das Git-Repository vor Angriffen zu schützen, läuft die Authentifizierung über kurzlebige Umgebungsvariablen ab, die nur im Arbeitsspeicher des Shell-Runners existieren und zustandslos sind. Auf Hypervisor-Ebene (Host:) startet der Befehl OpenTofu `tofu apply` zwei API-Operationen, die parallel laufen:

- **qm clone:** Hier wird die Struktur der virtuellen Festplatten anhand des versiegelten Golden Image (VM 9002) aufgebaut.
- **qm set (-initialization):** Dieser Schritt erzeugt dynamisch ein virtuelles CD-ROM-Medium (ConfigDrive2-ISO).

Um I/O-Leistungsprobleme auszuschließen und Datensicherheit zu gewährleisten, wird das Cloud-Init-ISO gezielt im ausfallsicheren ZFS-Spiegel-Pool (zpool0-mirror0) abgelegt und an die IDE-Schnittstelle 2 (ide2) der Ziel-VM angebunden.

Bei der anschließenden Boot-Phase nimmt das als Dienst laufende Cloudbase-Init im Windows-Gastsystem die Hardware-Injektion auf. Das UserDataPlugin liest die im ISO gespeicherte `ad-join.yaml` aus, entpackt sie und speichert sie als lokales PowerShell-Skript (`ad-join.ps1`) auf der Systempartition. Das Skript lädt dann automatisch mit Administratorrechten, was den automatischen Beitritt zur Active-Directory-Domäne `wiit-iac.local` und einen anschließenden Systemneustart ohne Administratoreingriff ermöglicht.

7. Qualitätssicherung, Testphase und Abnahme

7.1 Testprotokoll 1: End-to-End Zero-Touch Deployment

Testziel: Überprüfung, ob das Deployment nach dem Pipeline-Run ohne administrative Interaktionen bis zur Netzwerkerreichbarkeit und zur fehlerfreien Skript-Injektion erfolgreich verläuft.

Testdurchführung: Die Pipeline im GitLab-Repository wurde ausgelöst. Der Bootvorgang wurde parallel über die serielle Schnittstelle (COM0) in Proxmox überwacht, um die System- und Cloudbase-Init-Logdateien in Echtzeit zu prüfen.

Beobachtungen (Initialer Testlauf & Fehlerbehebung): Bei den ersten Testläufen trat ein kritischer Laufzeitfehler der Proxmox-API auf (TASK ERROR: volume 'local:snippets/ad-join.yaml' does not exist), der den Startvorgang der Instanz blockierte. Nach einer detaillierten Fehleranalyse zeigte sich, dass der lokale Storage-Pool des Hypervisors von Haus aus keine Skriptinhalte erlaubt. Erst nachdem der Inhaltstyp „Snippets“ explizit in der Storage-Konfiguration des Hypervisors autorisiert wurde, konnte die Pipeline das automatisierte Bereitstellungsskript korrekt injizieren.

Beobachtungen (Abschließender Testlauf): Nachdem das Berechtigungsproblem auf Storage-Ebene behoben war, wurde der Deployment-Prozess erneut gestartet.

- Das System startete fehlerfrei; die OOB-Phasen (Out-of-Box Experience) wurden durch die Sysprep-Versiegelung komplett übersprungen.
- Das Cloudbase-Init-Log bestätigte, dass das on-the-fly generierte ConfigDrive2-Medium erfolgreich erkannt wurde.
- Das SetHostNamePlugin hat den Computernamen in der Registry erfolgreich geändert.
- Das Netzwerk-Plugin hat dem paravirtualisierten VirtIO-Netzwerkadapter die richtige IP-Konfiguration aus dem IaC-Code zugewiesen.

Testergebnis: Bestanden. Nach der erfolgreichen Fehlerbehebung lief der Prozess völlig autonom ab. Nach genau 145 Sekunden war die neue Windows-Instanz stabil per ICMP (Ping) im vorgesehenen Management-VLAN erreichbar.

7.2 Testprotokoll 2: AD-Join und Client-Erreichbarkeit

Testziel: Validierung des automatisierten Domänenbeitritts im Active Directory, Überprüfung der DNS-Auflösung in der Provisionierungsphase sowie Sicherstellung der erforderlichen administrativen Berechtigungen.

Testdurchführung: Nach Abschluss der Bereitstellung und dem durch Cloudbase-Init ausgelösten Neustart wurde der Domänenbeitrittsstatus kontrolliert. Danach erfolgte ein erster RDP-Verbindungsaufbau (Remote Desktop Protocol) unter Verwendung von Domänen-Anmeldedaten. Beobachtungen (Erstes Testszenario & Fehlerbehandlung): Im ersten Durchlauf scheiterte das PowerShell-Skript ad-join.ps1 mit einer Fehlermeldung, wonach die Domäne wiit-iac.local nicht erreichbar war. Durch manuelle Untersuchung stellte sich heraus, dass die VM beim DHCP-Zuweis des Management-VLANs den internen Domain Controller nicht als primären DNS-Server zugewiesen bekommen hatte, was zu Namensauflösungsfehlern führte. Um das Problem zu beheben, wurde die main.tf modifiziert: Im initialization-Block der IaC-Konfiguration wurde das Argument dns { servers = ["10.6.232.10"] } (Adresse des Domain Controllers) direkt eingebunden. Dies veranlasst Proxmox dazu, das Windows-System beim Starten nur den internen DNS-Server für den AD-Join zu verwenden.

Beobachtungen (Finaler Testdurchlauf): Nach der Anpassung des IaC-Codes lief der Test problemlos ab: Das Computerobjekt DC-1010 wurde im Active Directory Domain Controller erfolgreich und ohne manuelle Eingriffe in der vorgesehenen Ziel-Organisationseinheit (OU=Servers,OU=ManagedDevices,DC=wiit-iac,DC=local) angelegt. Der RDP-Zugang mit einem dedizierten Standard-Domänenbenutzer (ohne lokale Admin-Privilegien) funktionierte sofort reibungslos. Beim RDP-Zugang traten keine SSL/TLS-Zertifikatswarnungen auf, da die Wurzelzertifikate der unternehmenseigenen Domänen-PKI durch die Gruppenrichtlinien (GPOs) automatisch während des Beitrittsvorgangs in den lokalen Zertifikatspeicher des Systems übernommen wurden.

Testergebnis: Erfolgreich.

7.3 Testprotokoll 3: Resilienz- und Rollback-Verfahren

Testziel: Nachweisen, dass sich Konfigurationsfehler im deklarativen Code schnell und ohne Datenverlust beheben lassen.

Testdurchführung: Ein fehlerhafter Commit wurde erstellt (Subnetzmaske /244 anstelle von /24). Danach erfolgte die Ausführung des Deployment-Laufs.

Beobachtungen:

- Cloudbase-Init brach die Netzwerkkonfiguration ab, da ein kritischer Syntaxfehler vorlag; die VM verlor ihre Netzwerkverbindung.
- Lösung: Mit git revert wurde der funktionierende Zustand sofort wiederhergestellt.
- Die defekte VM wurde mithilfe der Proxmox-API entfernt und anschließend ein neuer Klon-Prozess eingeleitet.

Testergebnis: Erfolgreich. Innerhalb von zwei Minuten stand eine vollständig funktionierende Instanz zur Verfügung. Das Verfahren unterstreicht die Robustheit des IaC-Paradigmas.

Parallel zum Code-Rollback wurde auch der Schutzmechanismus "State Reconciliation" von OpenTofu aktiviert und getestet. Sollte sich die Infrastruktur zwischen Erstellung des Ausführungsplans (tfplan) und dem eigentlichen Deployment ändern, lehnt die Pipeline das Deployment mit der aussagekräftigen Fehlermeldung "Error: Saved plan is stale" ab. Das demonstriert, dass die Architektur selbst bei gleichzeitigem, asynchronem Handeln mehrerer Administratoren vor unbeabsichtigten Überschreibungen und einer Beschädigung der State-Datei bewahrt bleibt.

8 Fazit und Ausblick

8.1 Soll-Ist-Vergleich und Zielerreichung

Der Soll-Ist-Vergleich dokumentiert, dass alle Projektziele vollumfänglich erreicht wurden:

Anforderungen des Auftraggebers (Soll)	Realisierte technische Umsetzung (Ist)	Erfüllungsgrad
IaC-Bereitstellung für Windows	Umgesetzt mittels Proxmox VE, GitLab CI und Terraform (bpg/proxmox-Provider).	100 %
Netzwerkisolation der Testumgebung	Aufgebaut als dedizierte, VLAN-isolierte Linux Bridge (vbr1) auf dem Hypervisor.	100 %
Zero-Touch Deployment	Erfolgreich implementiert. Cloudbase-Init verarbeitet Parameter über ConfigDrive2; die OOB wird durch unattend.xml automatisch übersprungen.	100 %
Automatisierter Active Directory Join	Gelöst durch ein PowerShell-Skript, das via YAML-Snippet eingespielt wird und eigenständig abläuft.	100 %

Anforderungen des Auftraggebers (Soll)	Realisierte technische Umsetzung (Ist)	Erfüllungsgrad
Versionierung & Rollback-Fähigkeit	Vollständig realisiert durch Git-Versionskontrolle und GitLab CI/CD-Integration.	100 %
Wirtschaftlichkeit (Amortisation)	Nachgewiesen. Break-Even wird nach 15 VM-Bereitstellungen erreicht (etwa 1,5 Monate Betriebszeit).	100 %
Einhaltung des IHK-Zeitbudgets (40 h)	Genau getroffen. Die Fertigstellung erfolgte präzise nach 40,0 Stunden.	100 %

Tabelle 7: Soll-Ist-Vergleich der Projektanforderungen.

8.2 Abnahmeprotokoll

Nach erfolgreicher Durchführung aller Testfälle wurde das Projekt der Systemadministration der WIIT AG im Zuge eines zweistündigen Abnahme-Workshops offiziell übergeben.

Prüfkriterium	Bewertung	Anmerkung
Template (VM 9002) ist vorhanden und startklar	Erfüllt	
Zero-Touch Deployment läuft ohne manuelle Eingriffe durch	Erfüllt	145 Sek. bis Erreichbarkeit
Hostname wird korrekt durch Cloudbase-Init gesetzt	Erfüllt	
Statische IP-Adresse wird korrekt zugewiesen	Erfüllt	VirtIO-Netzwerkadapter
Automatischer Active Directory Join erfolgreich	Erfüllt	OU=Servers, DC=wiit-iac, DC=local
RDP-Login mit Domänen-Benutzerkonto möglich	Erfüllt	
Netzwerkisolation der Testumgebung gewährleistet	Erfüllt	VLAN 2663, vmbr1
Git-Rollback funktioniert bei fehlerhafter Konfiguration	Erfüllt	git revert in < 2 Min.
Administrations-handbuch vollständig und verständlich	Erfüllt	
Zeitbudget von 40 Stunden eingehalten	Erfüllt	Exakt 40,0 h

Tabelle 8: Formelles Abnahmeprotokoll – Unterschrift Projektbetreuer & Prüfling erforderlich.

Ort, Datum

Projektverantwortlicher

Ort, Datum

Dragani

Projektbetreuer

8.3 Lessons Learned und persönliche Reflexion

Dieses Projekt hat im Kontext moderner Hybrid-Cloud-Infrastrukturen wertvolle Lehren vermittelt. Besonders zeitaufwändig und komplex erwies sich die Konfiguration sowie das Debugging von Cloudbase-Init zusammen mit Windows Sysprep. Während Linux standardmäßig über etablierte Verfahren zur Metadaten-Injektion verfügt, fehlen diese bei Windows vollständig, dies machte tiefgehende Auseinandersetzung mit den Mechanismen des Boot- und Anpassungsprozesses notwendig. Das knifflige Problem beim JSON-Parsing ließ sich schließlich durch den Aufbau kontinuierlicher Builds beheben und trug maßgeblich zur Stärkung der analytischen Fähigkeiten des Projektleiters bei. Mittels intelligenter Risikovorkehrungen und großzügig bemessenen Puffer gelang es, diese unerwartete Herausforderung zu meistern, ohne die geplanten Ressourcen zu überziehen. Aufschlussreich war darüber hinaus die Erfahrung mit „Golden Images“ (Templates) innerhalb deklarativer Prozesse. Praktische Versuche belegten, dass Hardware-Komponenten, die direkt im Template festgelegt sind (etwa fixierte Cloud-Init-Laufwerke oder hartcodierte VLAN-IDs), während automatischer Klonprozesse zu Race-Conditions und Abweichungen gegenüber der IaC-Orchestrierung führen können. Diese wichtige Erkenntnis resultiert in einer Kernforderung: Templates sollten maximal schlank und unkonfiguriert bleiben. Jegliche Hardware-bezogenen Festlegungen und Speicherverteilungen (wie die Zuweisung zu `zpool0-mirror0`) müssen ausschließlich durch IaC-Code gesteuert werden, damit eine zuverlässige State-Synchronisation erfolgt.

8.4 Ausblick und Skalierungspotenzial

Die aufgebaute IaC-Pipeline bietet ideale Grundlagen für kommende Automatisierungsprojekte. Ein naheliegender Fortschritt läge darin, Konfigurationsmanagement-Systeme wie Ansible oder SaltStack zu integrieren, die nach dem OS-Start automatisch applikationsspezifische Rollen (etwa IIS-Server, Datenbankinstanzen) installieren und konfigurieren könnten. Gleichzeitig ist die Methodik problemlos auf Linux-Betriebssysteme übertragbar, um innerhalb der WIIT AG eine einheitliche, betriebssystemübergreifende Multi-OS-Provisioning-Lösung aufzubauen. Mittel- bis langfristig könnte die Pipeline um Self-Service-Plattformen ergänzt werden, die es Entwicklern ermöglichen, eigenverantwortlich Testsysteme zu bestellen ganz ohne administrative Genehmigungsprozesse.

9 Anlagenverzeichnis und Anhänge

9.1 Technisches Glossar

Begriff	Definition
Active Directory (AD)	Zentraler Verzeichnisdienst von Microsoft zur Verwaltung von Benutzern, Gruppen und Computern in Domänennetzwerken.
Cloud-Init / Cloudbase-Init	Open-Source-Initialisierungswerkzeug für VMs. Cloudbase-Init ist der spezialisierte Port für Microsoft Windows.
ConfigDrive2	Vom Hypervisor on-the-fly erzeugtes virtuelles ISO-Image mit Metadaten (IP, Passwort, Hostname) für Cloudbase-Init.
Configuration Drift	Das schleichende Auseinanderdriften von Systemkonfigurationen durch unkoordinierte manuelle Eingriffe über einen längeren Zeitraum.
GitOps	Betriebliches Framework, das Git-Repositories als Single Source of Truth für deklarative Infrastrukturdefinitionen nutzt.
Infrastructure as Code (IaC)	Das softwarebasierte Verwalten und Bereitstellen von IT-Ressourcen über maschinenlesbaren Code statt über manuelle Klicks.
KVM	Kernel-based Virtual Machine – in Linux integrierter Typ-1-Hypervisor, der die technologische Basis von Proxmox VE bildet.
Proxmox VE	Open-Source-Virtualisierungsplattform auf Debian-Basis, die KVM-Hypervisoren und Linux-Container (LXC) integriert.
Sysprep	Microsoft-Werkzeug zur Systemvorbereitung, das maschinenspezifische SIDs löscht und das OS für das Klonen generalisiert.
Zero-Touch Deployment	Vollautomatisierter Bereitstellungsprozess, der nach dem Auslöser keinerlei manuelle Administratorinteraktion erfordert.

Tabelle 9: Technisches Glossar verwendeter Fachbegriffe.

9.2 Anlagen

1. Kundendokumentation
2. Selbständigkeitserklärung

9.3 Literaturverzeichnis

- [1] IHK Rhein-Neckar, 'Dokumentation der betrieblichen Projektarbeit IT-Berufe', online verfügbar: <https://www.ihk.de/rhein-neckar> (abgerufen: 17.05.2026).
- [2] IHK Köln, 'Formale Vorgaben für die IT-Projektdokumentation', online verfügbar: <https://www.ihk.de/koeln> (abgerufen: 17.05.2026).
- [3] Terraform Registry, 'proxmox_virtual_environment_vm | bpg/proxmox', online verfügbar: <https://registry.terraform.io/providers/bpg/proxmox/latest> (abgerufen: 17.05.2026).
- [4] Proxmox VE Wiki, 'Cloud-Init Support', online verfügbar: https://pve.proxmox.com/wiki/Cloud-Init_Support (abgerufen: 17.05.2026).
- [5] Cloudbase Solutions, 'cloudbase-init 1.1.9 documentation', online verfügbar: <https://cloudbase-init.readthedocs.io> (abgerufen: 17.05.2026).

- [6] Proxmox Support Forum, Bug ID 4493, 'Add full support for Proxmox · Issue #181 · cloudbase/cloudbase-init', online verfügbar: <https://github.com/cloudbase/cloudbase-init/issues> (abgerufen: 17.05.2026).
- [7] BSI, 'IT-Grundschutz-Kompendium', online verfügbar: <https://www.bsi.bund.de> (abgerufen: 17.05.2026).

9.2 Literaturverzeichnis

- [1] IHK Rhein-Neckar, 'Dokumentation der betrieblichen Projektarbeit IT-Berufe', online verfügbar: <https://www.ihk.de/rhein-neckar> (abgerufen: 17.05.2026).
- [2] IHK Köln, 'Formale Vorgaben für die IT-Projektdokumentation', online verfügbar: <https://www.ihk.de/koeln> (abgerufen: 17.05.2026).
- [3] Terraform Registry, 'proxmox_virtual_environment_vm | bpg/proxmox', online verfügbar: <https://registry.terraform.io/providers/bpg/proxmox/latest> (abgerufen: 17.05.2026).
- [4] Proxmox VE Wiki, 'Cloud-Init Support', online verfügbar: https://pve.proxmox.com/wiki/Cloud-Init_Support (abgerufen: 17.05.2026).
- [5] Cloudbase Solutions, 'cloudbase-init 1.1.9 documentation', online verfügbar: <https://cloudbase-init.readthedocs.io> (abgerufen: 17.05.2026).
- [6] Proxmox Support Forum, Bug ID 4493, 'Add full support for Proxmox · Issue #181 · cloudbase/cloudbase-init', online verfügbar: <https://github.com/cloudbase/cloudbase-init/issues> (abgerufen: 17.05.2026).
- [7] BSI, 'IT-Grundschutz-Kompendium', online verfügbar: <https://www.bsi.bund.de> (abgerufen: 17.05.2026).